Stewer Gabriel

DOCUMENTAÇÃO DO SISTEMA

Documentação *completa*

Requisitos, arquitetura, banco de dados, API, testes, segurança e manual do painel do site do Stewer Gabriel.

Site e painel de stewergabriel.com
Visagismo e consultoria de imagem · Palmas, Tocantins · Desenvolvido por Gustavo Dev

Versão 2.2.0
08 de novembro de 2026

Sumário

01 Requisitos

Atores, restrições, requisitos funcionais e não funcionais.

02 Arquitetura

Camadas, como a página abre, mapa, fotos, edição simultânea, segurança e decisões técnicas.

03 Modelo do banco

Diagrama entidade-relacionamento, tabelas, regras e migrations.

04 Referência da API

Rotas, formatos de pedido e resposta, erros e sessão.

05 Plano e relatório de testes

Caixa branca, cinza e preta, segurança, desempenho, rastreabilidade e achados.

06 Segurança

Os controles pelo OWASP Top 10 e onde cada um é testado.

07 Manual do painel

Como o Stewer usa o painel, em linguagem simples.

CAPÍTULO 01

Requisitos

Atores, restrições, requisitos funcionais e não funcionais.

Site institucional do Stewer Gabriel (visagista e consultor de imagem masculina em Palmas, TO) e o painel em `/admin` para ele mudar o conteúdo sozinho, sem depender de quem programou.

Cada requisito tem um identificador. Os testes citam esse identificador no nome (`RF11 · …`), e a matriz de rastreabilidade liga um ao outro.

Atores

ATOR	QUEM É	O QUE FAZ
Visitante	qualquer pessoa na internet	vê o trabalho, chama no WhatsApp, abre a rota no mapa
Stewer	quem tem login no painel (ele e quem ele der acesso)	mantém textos, fotos, listas, contatos e mapa; dá e tira acesso
Mantenedor	quem cuida do código (hoje, o Gustavo, pelo GitHub)	publica, aplica migrations, cria o primeiro acesso, troca vídeos

Restrições

ID	RESTRIÇÃO
RE01	Custo zero de operação: só o domínio é pago.
RE02	Hospedagem no Cloudflare (Pages, Functions e D1 no plano gratuito), no mesmo projeto do site.
RE03	O design aprovado não muda: o site lido do banco fica igual ao que já estava no ar.
RE04	O Stewer não é técnico: tudo pelo painel, em português, também pelo celular.
RE05	Vídeos ficam no código: não cabem numa linha do D1 e um serviço de arquivos (R2) exigiria cartão na conta.

Requisitos funcionais

Site público

ID	REQUISITO
RF01	O site mostra as fotos da abertura, os números, os serviços, os itens da consultoria, os tópicos do ensino, as fotos das aulas e o portfólio vindos do banco, na ordem do painel.
RF02	Item oculto no painel não aparece. Seção de serviços sem nenhum item visível sai da página; números, consultoria, tópicos e galeria das aulas escondem só o bloco; portfólio vazio mostra o aviso; abertura sem foto visível mantém as fotos de reserva.
RF03	Se a API falhar ou demorar mais de 2,5 s, o site abre com o conteúdo de reserva do código.
RF04	Textos e fotos fixos de cada seção (abertura, quem é, serviços, consultoria, ensino, portfólio, onde atende, contato, rodapé) vêm do banco quando foram trocados no painel.
RF05	WhatsApp (botões e telefone), mensagem pronta, Instagram e o local do mapa (marcador, coordenadas, rota e Google Maps) são os configurados no painel.
RF06	Os vídeos do portfólio e das aulas continuam no código e aparecem intercalados com as fotos do banco.
RF07	O mapa nunca mostra aviso de chave: usa o fundo escuro da CARTO quando há chave; sem ela, ou se falhar, o OpenFreeMap escuro; sem ele (ou sem WebGL), o OpenStreetMap escurecido. O mapa só é baixado quando a pessoa chega perto dele.
RF08	A cada carregamento, uma abertura de marca rápida desenha o logo e o nome e sobe revelando o site, sem nunca prender o site. Ao rolar, títulos, fotos e números entram com movimento. Quem pede menos movimento no sistema vê tudo parado, sem abertura.
RF09	Um link com âncora (por exemplo `#onde-atende`) abre direto na seção.
RF21	No celular, um dedo sobre o mapa rola a página; o mapa se move com dois dedos, e quem tenta arrastar com um dedo vê o aviso.

Painel

ID	REQUISITO
RF10	Só entra no painel quem tem login; a sessão expira em 8 horas.
RF11	O Stewer cria, edita, apaga, oculta e reordena as sete listas: fotos da abertura, números, serviços, consultoria, ensino, fotos das aulas e portfólio.
RF12	O Stewer envia fotos, corrige a descrição, apaga as que não estão em uso e baixa uma ou todas (.zip).
RF13	Fotos enviadas são reduzidas (até 1600px) e convertidas para WebP no navegador, antes de subir.
RF14	O Stewer edita os textos e fotos fixos de cada seção e pode voltar qualquer campo ao original.
RF15	O Stewer troca WhatsApp, mensagem pronta, Instagram, o endereço e horário, e o local do mapa (colando o link do Google Maps, inclusive o curto do botão Compartilhar, ou as coordenadas, e ajustando o alfinete no minimapa), e a própria senha.
RF16	Quem tem acesso dá e tira acesso de outra pessoa e gera senha temporária para quem esqueceu.
RF17	Quem entra com senha temporária é obrigado a criar uma senha própria antes de usar o painel.
RF18	Toda alteração e todo login ficam registrados (quem, o quê, quando) e são consultáveis por dia e pessoa.
RF19	O painel mostra o espaço usado do banco e avisa a partir de 70%.
RF20	Nas listas de foto, escolher ou enviar a foto já cria o item no fim da lista, com o enquadramento padrão.

Requisitos não funcionais

ID	CATEGORIA	REQUISITO
RNF01	Segurança	Senhas guardadas só como hash PBKDF2-SHA256 com salt por usuário.
RNF02	Segurança	Sessão em cookie `HttpOnly`, `SameSite=Strict` (e `Secure` em HTTPS), assinada (JWT, só HS256) e gravada no servidor: "Sair" encerra a sessão, trocar a senha encerra as outras, senha temporária e acesso removido encerram todas.
RNF03	Segurança	Pedido que altera dados vindo de outra origem é recusado (CSRF). Páginas e API com cabeçalhos de segurança (CSP sem script inline, `nosniff`, sem iframe de outro site, HSTS).
RNF04	Segurança	Login trava 15 minutos depois de 5 senhas erradas; a mensagem não revela se o e-mail existe.
RNF05	Segurança	Nenhum nome de tabela ou coluna vem do pedido; valores sempre por bind (sem SQL injection).
RNF06	Segurança	Upload aceita só JPG, PNG e WebP conferidos pelos bytes, até 1,5 MB.
RNF07	Integridade	Duas pessoas editando o mesmo item: quem salva por cima recebe aviso, nada é sobrescrito em silêncio.
RNF08	Integridade	Mudanças de vários registros (reordenar, contatos, textos de uma seção) são gravadas juntas ou nenhuma.
RNF09	Integridade	Foto em uso não pode ser apagada; dados inválidos são recusados com mensagem em português.
RNF10	Contrato	As respostas da API seguem o formato que o site e o painel esperam (schemas em `tests/contratos.ts`).
RNF11	Usabilidade	Painel em português, utilizável no celular (390px) e no computador.
RNF12	Acessibilidade	Painel sem violações graves (axe, WCAG 2 A/AA); o conteúdo que o painel coloca no site não cria violação nova.
RNF13	Capacidade	Cabe no plano gratuito do D1 (500 MB): fotos reduzidas, tentativas de login limpas sozinhas, auditoria guardada por um ano.
RNF14	Disponibilidade	O site público nunca fica em branco por causa do backend (ver RF03).
RNF15	Manutenção	Esquema do banco versionado em migrations; cada parte testada automaticamente no CI.
RNF16	Fidelidade	O site montado com o conteúdo do banco é visualmente igual ao site anterior ao painel.
RNF17	Segurança	O servidor só abre links do Google (link curto do mapa), por HTTPS, sem usuário nem porta, com no máximo 6 redirecionamentos (sem SSRF). Dependências sem vulnerabilidade conhecida em produção.

ID	CATEGORIA	REQUISITO
RNF18	Desempenho	Core Web Vitals "bons" do Google no celular com 4G lento: LCP ≤ 2,5 s, CLS ≤ 0,1, TBT ≤ 200 ms, INP ≤ 200 ms. Abertura do site com até 65 KB de JavaScript; nada de outro servidor na abertura. Lighthouse ≥ 90 em desempenho e ≥ 95 no resto.
RNF19	Desempenho	`/api/conteudo` com p95 ≤ 300 ms; a API aguenta 100 conexões simultâneas sem erro 5xx e uma força bruta no login sem cair.

Fora do escopo

- Upload de vídeo pelo painel (ver RE05). Para trocar um vídeo, o arquivo vai para `public/videosImgs/` (portfólio) ou `public/instrutorVideos/` (aulas) e o caminho entra em `VIDEOS_PORTFOLIO` ou `VIDEOS_AULAS` em `src/content/site.ts`.
- Agendamento online: fica no app de agenda, que é outro projeto.
- Trocar cores, fontes e layout: o painel muda o conteúdo, não o design.

CAPÍTULO 02

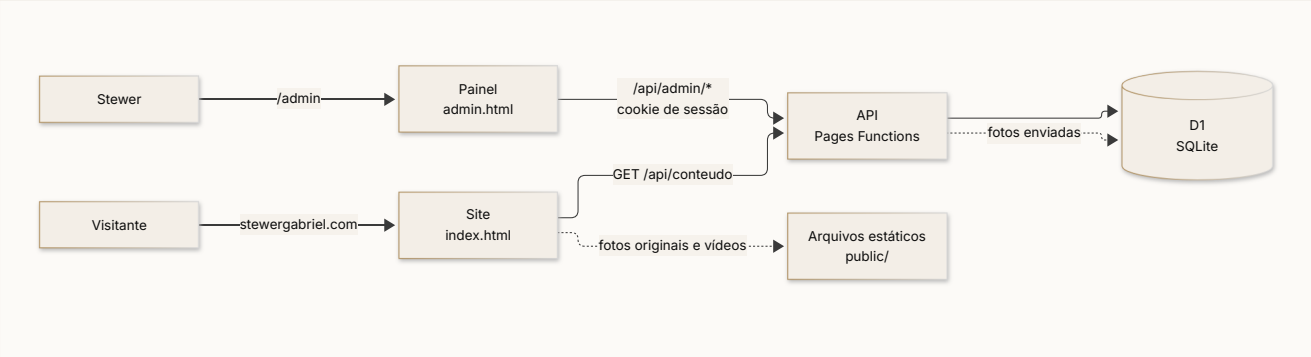
Arquitetura

Camadas, como a página abre, mapa, fotos, edição simultânea, segurança e decisões técnicas.

O site e o painel são um projeto só, publicado no Cloudflare Pages. O mesmo deploy leva três coisas:

- o **site** (`index.html` , React + Vite + Tailwind), que é o que o cliente vê;
- o **painel** (`admin.html` , React), uma segunda entrada do build, em `/admin` ;
- a **API** (`functions/` , TypeScript nas Pages Functions), que fala com o banco **D1** (SQLite gerenciado pela Cloudflare).

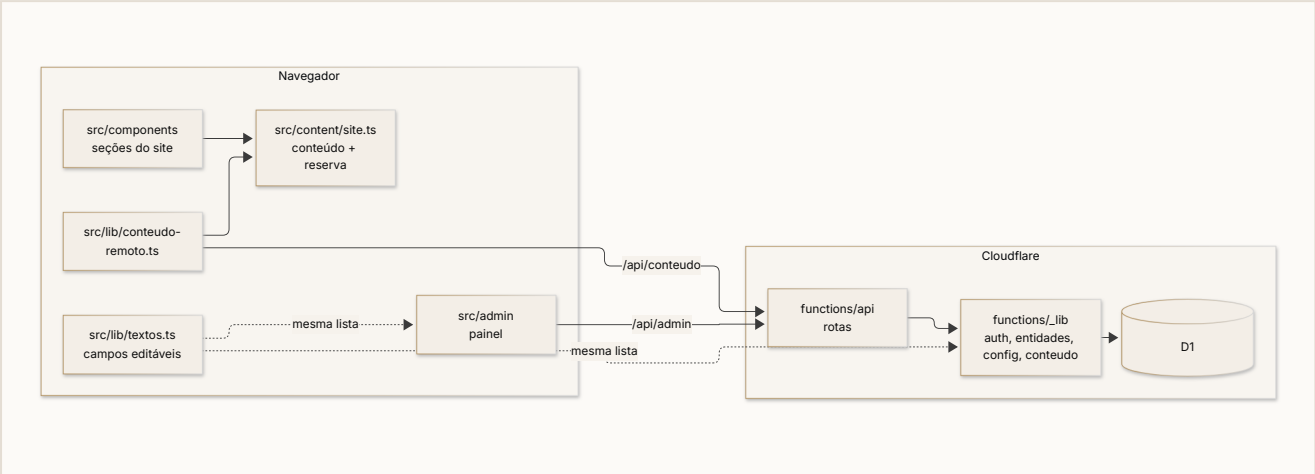
Tudo no plano gratuito. Não existe servidor ligado esperando: cada pedido à API acorda uma função na borda da Cloudflare, que responde em milissegundos.



Hospedagem e quem cuida de cada parte

PARTE	ONDE FICA	QUEM MEXE
Código	GitHub (`Gustavo16378/stewer-site`)	o mantenedor
Site, painel e API	Cloudflare Pages, projeto `stewer-gabriel`	deploy automático a cada push na `main`
Banco	Cloudflare D1 `stewer-site`	o painel (conteúdo) e as migrations (estrutura)
Segredo das sessões	variável `JWT_SECRET` do Pages, tipo Secret	o mantenedor, uma vez

Camadas



- `src/content/site.ts` guarda todo o conteúdo do site, com o texto original como reserva. Os componentes só leem daqui.
- `src/lib/conteudo-remoto.ts` busca `/api/conteudo` antes do primeiro render e troca o conteúdo de `site.ts` no lugar.
- `functions/_lib/entidades.ts` descreve as sete listas (tabela, campos, limites). Uma rota só (`/api/admin/[entidade]`) atende todas.
- `functions/_lib/config.ts` valida contatos e coordenadas.
- `functions/_lib/auth.ts` cuida de senha (PBKDF2) e sessão (JWT).

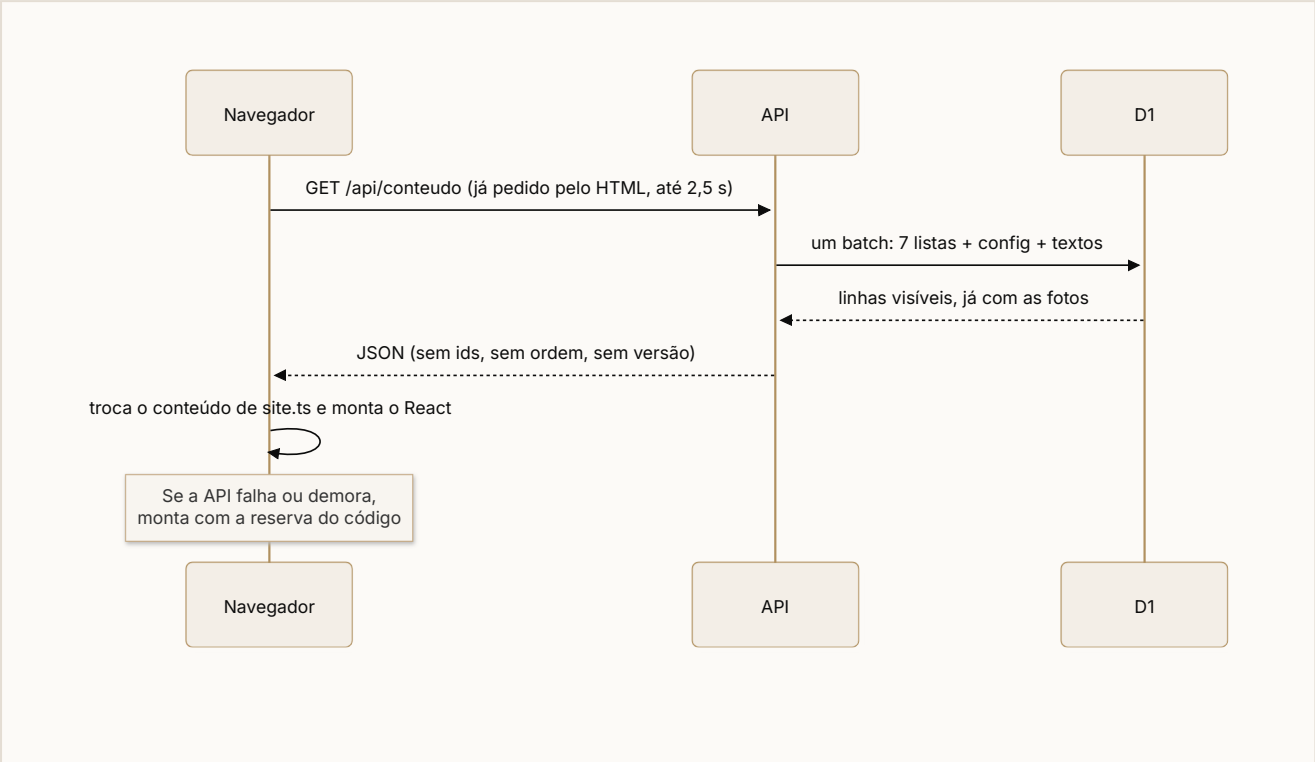
Uma fonte só para o que é compartilhado

`src/lib/textos.ts` é a lista de todos os textos fixos que podem ser trocados: chave, seção, rótulo, tipo e limite. Três lugares usam a mesma lista:

1. o **painel** monta a aba "Textos do site" a partir dela;
2. a **API** só aceita chaves que estão nela, com os limites dela;
3. o **site** aplica os valores salvos com a função `gravar` de cada campo.

Um campo novo entra num lugar e aparece nos três.

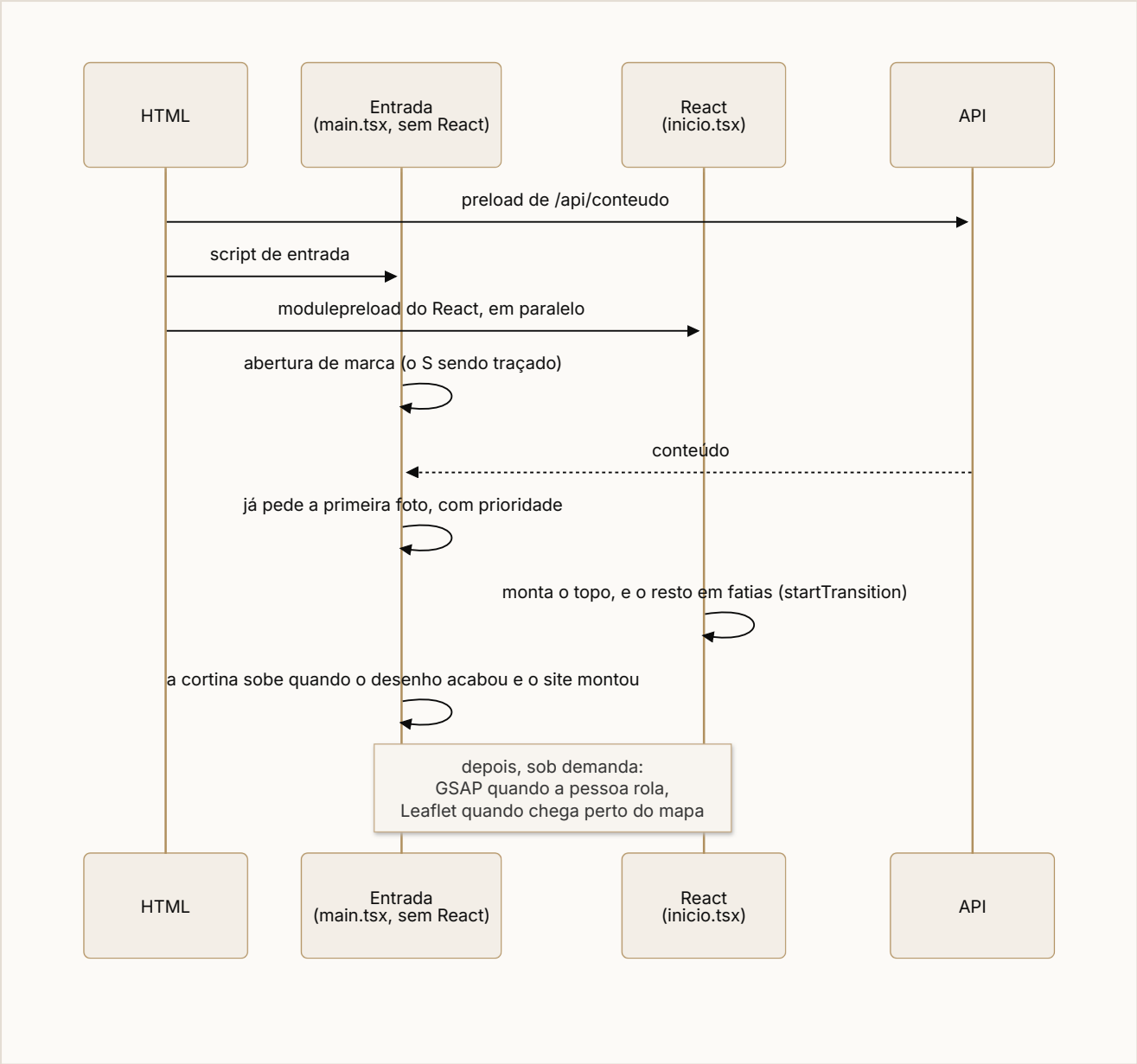
Fluxo de leitura do site



Trocar o conteúdo antes do primeiro render, e não depois com estado, é de propósito: o mapa e as animações de entrada leem o conteúdo uma vez, na montagem. O resultado é que o site com o banco fica idêntico ao de antes, pixel por pixel (conferido comparando capturas da página inteira no computador e no celular).

Como a página abre (desempenho)

A meta é a do Google para um celular com 4G lento: a maior coisa da tela (a foto da abertura) em até 2,5 s, nada pulando de lugar e o processador livre para responder. Medido a cada push (ver testes).



- **A entrada não tem React** (`src/main.tsx`): roda assim que chega, mostra a abertura de marca, pega o conteúdo e começa a baixar a primeira foto, enquanto o React chega em paralelo (`src/inicio.tsx` , pré-carregado por um plugin no `vite.config.ts`).
- **Abertura de marca** (`src/preloader.ts` e `.css`): o S do logo (vetor em `src/lib/logo.ts`) traçado com `stroke-dashoffset` , em CSS puro. Cobre a espera pelo conteúdo em vez de somar a ela. A cada carregamento; nunca para quem pede menos movimento; nunca prende o site (sobe em até 6 s).
- **O resto da página** monta numa transição do React, em fatias: montar tudo de uma vez travava o processador do celular por ~200 ms.
- **Movimento de rolagem** (`src/lib/movimento.ts`): GSAP com ScrollTrigger e SplitText, ligado por atributos no HTML (`data-revelar` , `data-contar` , `data-parallax`). Só é baixado quando a pessoa começa a rolar.
- **Mapa** (`src/lib/mapa-site.ts`): o Leaflet só é baixado a 600 px da seção.
- **Fontes** servidas pelo próprio site (`@fontsource`), só o subconjunto latino e os pesos usados.

Mapa

O fundo escuro do mapa tenta três fontes, nesta ordem (`src/lib/mapa.ts`):

1. **CARTO** `dark_all`, o visual original do design, quando há chave em `src/lib/mapa-fontes.ts` (desde setembro de 2026 a CARTO exige chave);
2. **OpenFreeMap** escuro, em vetor (MapLibre), sem chave nenhuma;
3. **OpenStreetMap** padrão escurecido por filtro, para quando o OpenFreeMap está fora ou o aparelho não tem WebGL.

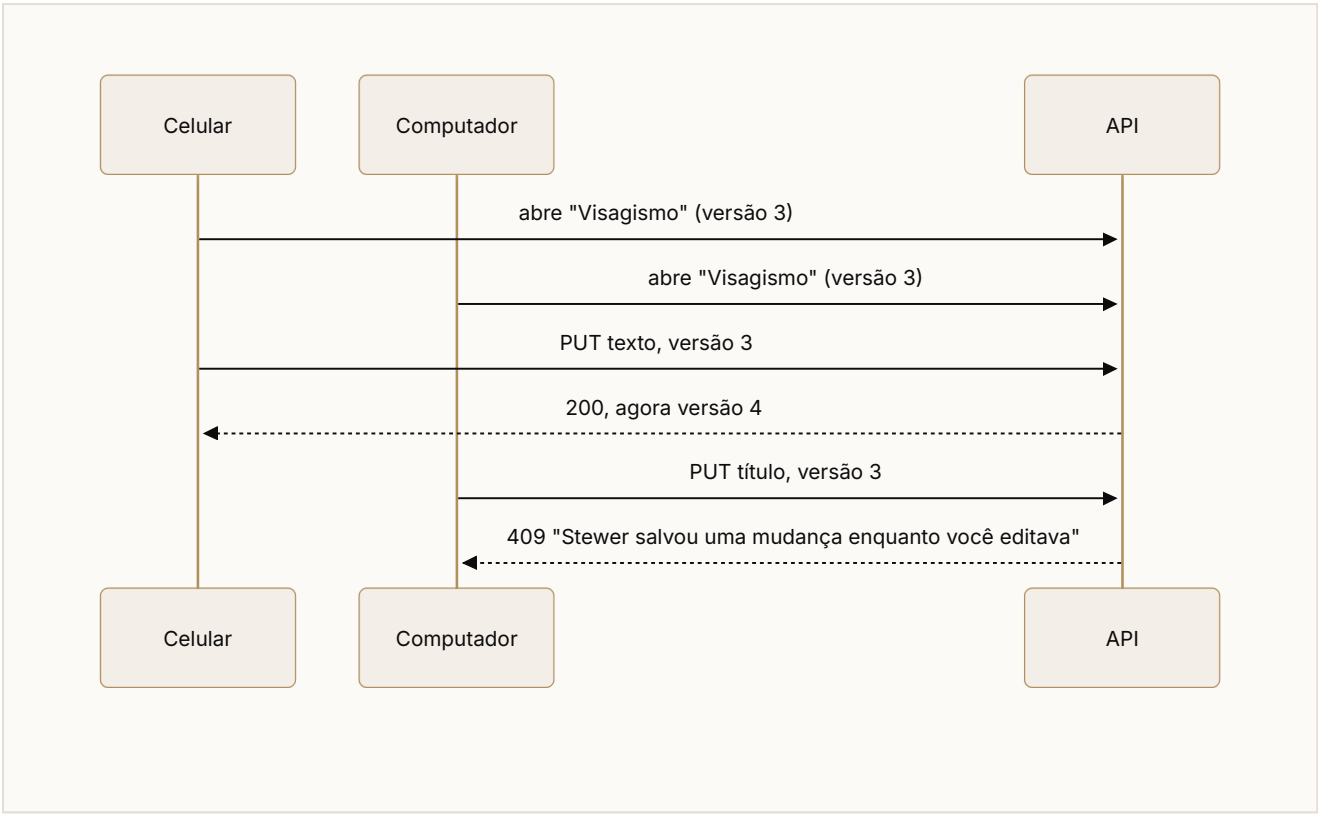
A troca é automática quando os pedaços do mapa falham. O mesmo fundo vale para o minimapa do painel, onde o Stewer cola o link do Google Maps ou arrasta o alfinete.

Fotos e vídeos

- **Fotos originais** continuam em `public/` e entram no banco só como endereço (`midias.caminho`). Não podem ser apagadas pelo painel.
- **Fotos novas** são reduzidas no navegador (até 1600px, WebP, até 1,4 MB) e guardadas dentro do D1 (`midias.dados`). A API entrega em `/api/midias/:id` com cache de um ano, porque o conteúdo de um id nunca muda: trocar a foto cria outro id.
- **Vídeos** ficam no código (`VIDEOS_PORTFOLIO` e `VIDEOS_AULAS`), porque não cabem numa linha do D1. O site intercala foto, vídeo, foto...

Edição simultânea

Cada item de lista tem uma coluna `versao`. O painel manda a versão que tinha quando abriu o item; a API grava com `WHERE versao = ?`. Se outra pessoa salvou no meio, a gravação não acha a linha e a API responde 409 dizendo quem salvou antes. Ninguém perde a mudança do outro sem saber.



O interruptor "No site" não manda versão (vale o último), porque é uma troca de um campo só e o resultado é sempre o que a pessoa vê.

Segurança

O resumo por risco do OWASP Top 10, com o teste de cada controle, está em `SECURITY.md`.

- Senha: PBKDF2-SHA256, 100 mil iterações, salt por usuário. O hash é calculado igual na API e no script de criar acesso.
- Sessão: JWT HS256 no cookie `sg_sessao`, `HttpOnly`, `SameSite=Strict`, `Path=/api`, `Secure` em HTTPS, 8 horas, apontando para uma linha da tabela `sessoes`. "Sair" apaga a linha; trocar a senha apaga as outras; senha temporária e acesso removido apagam todas.
- CSRF: além do `SameSite`, todo pedido que muda dados precisa vir da própria origem.
- Login trava por 15 minutos depois de 5 erros no mesmo e-mail.
- SQL: nomes de tabela e coluna só da lista fechada de `entidades.ts`; valores sempre por bind.
- Upload: tipo conferido pelos bytes, não pelo que o navegador diz. SVG é recusado (pode carregar script).
- Conteúdo: o React escapa todo texto; o único HTML montado à mão (o rótulo do marcador do mapa) passa por escape. Testado no E2E.
- Cabeçalhos: CSP sem script inline nem origem solta, `nosniff`, sem iframe de outro site, HSTS (`public/_headers` para as páginas, `functions/_lib/http.ts` para a API).
- SSRF: o link curto do mapa é aberto pelo servidor, só se for do Google.
- Dependências: `npm audit` na CI, Dependabot e CodeQL.

Decisões

DECISÃO	POR QUÊ
Pages Functions + D1, não um back em Java ou Node num servidor	servidor gratuito dorme e demora quase um minuto para acordar; aqui o custo é zero e a resposta é imediata
Mesmo repositório e mesmo deploy do site	um push publica tudo junto; sem CORS, sem segundo domínio
Fotos no D1, não no R2	o R2 pede cartão cadastrado na conta; foto reduzida cabe numa linha
Vídeos no código	não cabem no D1; o Stewer posta vídeo bem menos que foto
Conteúdo trocado antes do render	mantém o site idêntico ao original, sem piscar
Painel com a identidade do site	o Stewer reconhece a própria marca; o painel não parece um sistema genérico
Uma rota para as sete listas	as regras de cada campo moram num lugar só e são testadas uma vez
Sessão no banco, além do JWT	um JWT sozinho não tem como ser revogado: "Sair" precisa valer de verdade
Abertura de marca em CSS, não GSAP	aparece em ~300 ms, antes do React e do GSAP chegarem, e cobre a espera em vez de somar
GSAP só quando a pessoa rola	antes disso nada abaixo da dobra aparece; baixar e rodar o GSAP na abertura travava o celular ~250 ms
Fontes no próprio site	o CSS do Google Fonts bloqueava a primeira pintura e mandava o IP do visitante ao Google
Três fontes para o fundo do mapa	a CARTO passou a exigir chave e o site mostrava "API KEY REQUIRED"; agora nunca fica sem mapa

CAPÍTULO 03

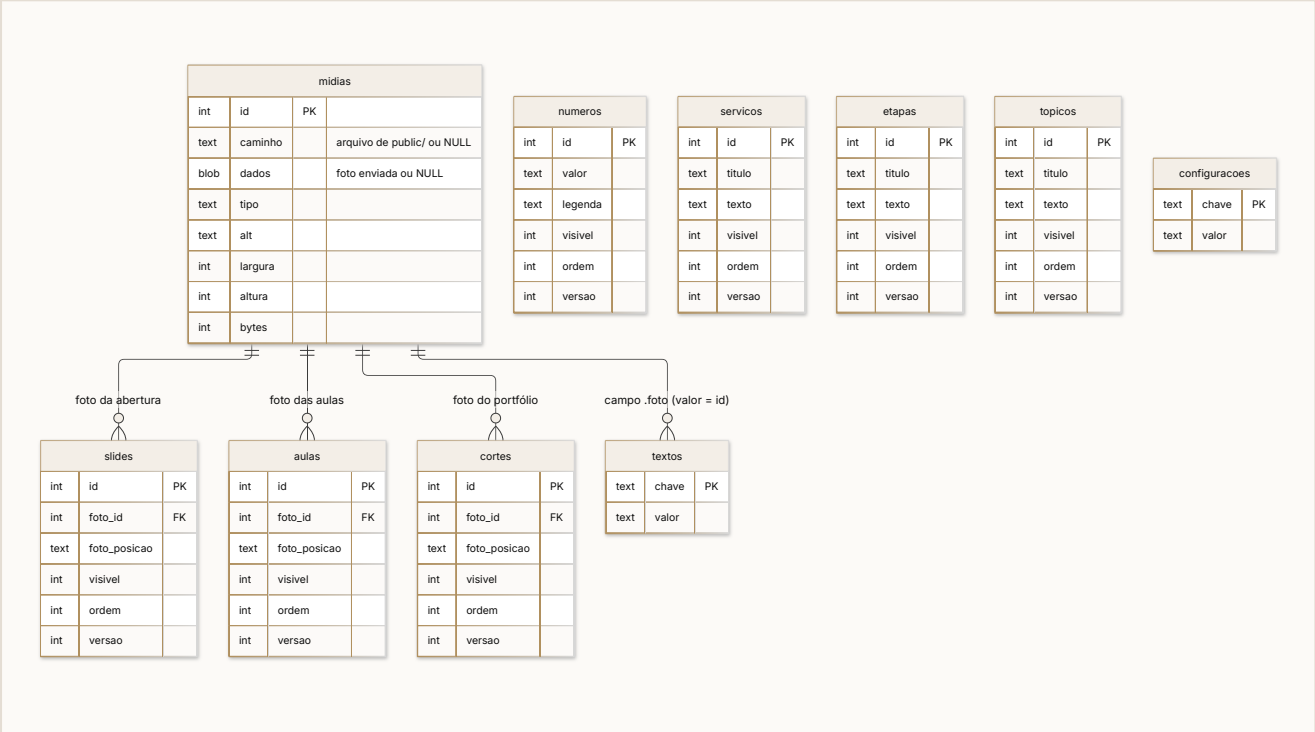
Modelo do banco

Diagrama entidade-relacionamento, tabelas, regras e migrations.

Cloudflare D1 (SQLite). O esquema está em `migrations/0001_schema.sql` e o conteúdo inicial em `migrations/0002_seed.sql`, gerado por `scripts/gerar-seed.ts`.

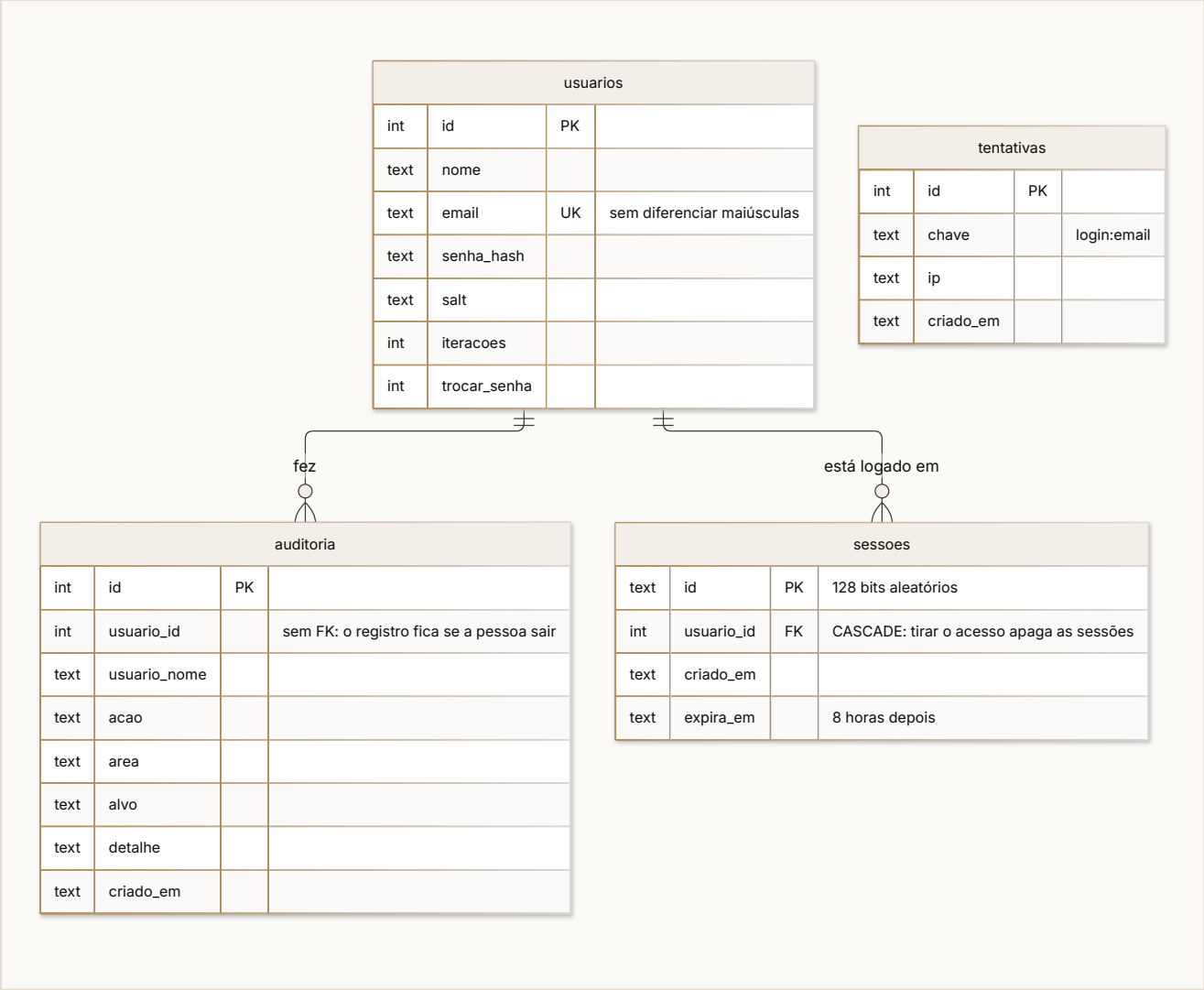
Diagrama entidade-relacionamento

Conteúdo do site



`textos` aponta para `midias` só nas chaves que terminam em `.foto`, sem chave estrangeira (o valor é texto). Por isso a API confere a foto antes de gravar e conta esse uso antes de deixar apagar uma foto.

Painel e operação



Tabelas

TABELA	O QUE GUARDA	APARECE NO SITE EM
`midias`	toda foto, original ou enviada	onde for usada
`slides`	fotos que se revezam na abertura	topo
`numeros`	+800 clientes, 10+ anos…	Quem é
`servicos`	cards numerados	Serviços
`etapas`	itens da consultoria	Consultoria
`topicos`	o que se aprende	Ensino
`aulas`	galeria dos bastidores	Ensino
`cortes`	fotos do portfólio	Portfólio
`textos`	textos e fotos fixos trocados no painel	todas as seções
`configuracoes`	`whatsapp`, `instagram`, `mensagem`, `latitude`, `longitude`	botões, contato, mapa
`usuarios`	quem entra no painel	não aparece
`sessoes`	cada login aberto (o token do cookie aponta para uma linha)	não aparece
`tentativas`	freio de login	não aparece
`auditoria`	quem fez o quê	não aparece

Regras que o próprio banco garante

Mesmo que a API tivesse um bug, o banco recusa:

- `visivel` e `trocar_senha` fora de 0 e 1 (CHECK);
- foto sem arquivo e sem bytes, ou com dimensão zero (CHECK);
- item de lista de foto sem foto (NOT NULL) ou apontando para foto que não existe (FOREIGN KEY);
- apagar uma foto em uso por uma lista (FOREIGN KEY com RESTRICT);
- e-mail repetido, sem diferenciar maiúsculas (UNIQUE com NOCASE);
- ação de auditoria fora da lista (CHECK);
- sessão de pessoa que não existe (FOREIGN KEY), e as sessões somem junto com a pessoa (CASCADE).

Migrations

ARQUIVO	O QUE FAZ
`0001_schema.sql`	cria todas as tabelas e índices
`0002_seed.sql`	conteúdo que já estava no site, gerado do código
`0003_sessoes.sql`	sessões no servidor: "Sair" e troca de senha encerram o token
`0004_foto_leve.sql`	a foto `corte5` passa de PNG (871 KB) a JPEG (96 KB)

Regra de ouro (detalhada no CONTRIBUTING): **migration nova nunca altera uma antiga** e é aplicada em produção antes do código que depende dela entrar na `main`.

```
npm run db:local    # banco de desenvolvimento (.wrangler/)
npm run db:remoto   # produção
```

Capacidade

O plano gratuito do D1 aceita 500 MB por banco. Quase todo o peso são as fotos enviadas, cerca de 0,3 MB cada depois da redução: dá espaço para mais de mil fotos. Texto quase não ocupa. As tentativas de login com mais de um dia são apagadas a cada login, as sessões vencidas também, e a auditoria guarda um ano. O painel mostra o uso em "Minha conta" e avisa no topo a partir de 70%.

CAPÍTULO 04

Referência da API

Rotas, formatos de pedido e resposta, erros e sessão.

Todas as rotas ficam em `/api` e rodam nas Pages Functions, no mesmo endereço do site. O formato exato de cada resposta está nos schemas de `tests/contratos.ts`, conferidos a cada rodada de testes.

Convenções

- Pedidos com corpo mandam `Content-Type: application/json` (exceto envio de foto, que é `multipart/form-data`). Outro tipo dá **415**.
- Corpo grande demais dá **413** (64 KB no geral, 32 KB nos textos, 2 KB em login, senha e contatos).
- Todo erro sai como `{ "erro": "mensagem em português" }`, pronta para mostrar na tela. Erro inesperado sai como 500 sem detalhe interno.
- Nenhuma resposta da API fica em cache (`Cache-Control: no-store`), exceto as fotos e `/conteudo` (`no-cache`: o navegador sempre confere com o servidor, mas pode reaproveitar o `<link rel="preload">` do `index.html`).
- Toda resposta leva os cabeçalhos de segurança (`nosniff`, `X-Frame-Options: DENY`, CSP `default-src 'none'`), inclusive erro e foto.
- Rota que não existe responde **404** (ou **405** para método diferente de GET) em JSON, nunca a página do site.
- Datas vêm do D1 em UTC, no formato `2026-11-07 22:30:00`.

STATUS	QUANDO
400	dado inválido (a mensagem diz qual)
401	sem sessão, sessão vencida ou senha errada
403	outra origem (CSRF), ou senha temporária ainda não trocada
404	rota, lista ou item que não existe (id de rota só com dígitos)
405	método que a rota não aceita
409	conflito: alguém salvou antes, foto em uso, lista mudou ao reordenar
413 / 415	corpo grande demais / tipo errado
429	login travado depois de 5 erros
502	o Google não respondeu ao abrir o link curto do mapa

Público

`GET /conteudo`

Tudo que o site mostra, numa ida só ao banco. Só itens visíveis, na ordem do painel, sem ids, ordem nem versão.

```
{
  "slides":   [{ "src": "/photos/Stewer1.jpg", "alt": "Stewer Gabriel", "largura": 1024,
"altura": 1024, "posicao": "50% 0%" }],
  "numeros":  [{ "valor": "+800", "legenda": "Clientes atendidos por ano" }],
  "servicos": [{ "titulo": "Corte", "texto": "Cada corte é…" }],
  "etapas":   [{ "titulo": "Análise de Visagismo", "texto": "…" }],
  "topicos":  [{ "titulo": "Técnicas de Corte", "texto": "…" }],
  "aulas":    [{ "src": "/instrutorImgs/instrutor2.jpeg", "alt": "…", "largura": 960, "altura":
1280, "posicao": "50% 50%" }],
  "cortes":   [{ "src": "/api/midias/12", "alt": "Degradê com risca", "largura": 1200,
"altura": 1600, "posicao": "50% 50%" }],
  "config":   { "whatsapp": "5563985116783", "instagram": "stewer.gabriel", "mensagem": "Olá
Stewer, …", "latitude": "-10.18256053", "longitude": "-48.32910908" },
  "textos":   { "hero.lema": "Novo lema", "sobre.foto": { "src": "/api/midias/9", "alt": "…",
"largura": 800, "altura": 1000 } }
}
```

`textos` só traz os campos trocados no painel; o resto o site já tem.

`GET /midias/:id`

A foto. Foto enviada pelo painel sai com o tipo real, `nosniff` e cache de um ano (`immutable`). Foto original redireciona (302) para o arquivo em `public/`. `?baixar=1` entrega como download, com nome legível tirado da descrição (`degrade-com-risca-12.webp`).

Sessão

`POST /auth/login`

`{ "email", "senha" }` → `{ "usuario": { id, nome, email, trocarSenha } }` e o cookie `sg_sessao`. Senha errada e e-mail que não existe dão a mesma resposta (401). Depois de 5 erros no mesmo e-mail, 429 por 15 minutos.

O cookie leva um JWT (HS256, 8 horas) que aponta para uma linha da tabela `sessoes`. Sem a linha, o token não vale, por melhor que seja a assinatura.

`POST /auth/logout`

Apaga o cookie e encerra a sessão no servidor: um token copiado antes deixa de valer. As sessões da mesma pessoa em outros aparelhos continuam.

`GET /auth/eu`

Quem está logado, ou 401. O painel usa ao abrir.

Painel (`/admin`, exige sessão)

Toda rota de `/api/admin` passa pelo `_middleware.ts`: sem sessão, 401; pedido que muda dados vindo de outra origem, 403; pessoa removida perde o acesso na hora; senha temporária só deixa trocar a senha.

Listas: `slides`, `numeros`, `servicos`, `etapas`, `topicos`, `aulas`, `cortes`

MÉTODO	ROTA	O QUE FAZ
GET	`/admin/:lista`	tudo, inclusive oculto, na ordem
POST	`/admin/:lista`	cria no fim; responde 201 com o item
PUT	`/admin/:lista/:id`	altera só os campos enviados
DELETE	`/admin/:lista/:id`	apaga
PUT	`/admin/:lista/ordem`	`{ "ids": [3, 1, 2] }`, todos os itens, numa transação

Campos de cada lista:

LISTA	CAMPOS	LIMITES
`servicos`	`titulo`, `texto`, `visivel`	40 / 300
`etapas`	`titulo`, `texto`, `visivel`	50 / 300
`topicos`	`titulo`, `texto`, `visivel`	50 / 200
`numeros`	`valor`, `legenda`, `visivel`	10 / 40
`slides`, `aulas`, `cortes`	`foto_id`, `foto_posicao` (`"50% 30%"`), `visivel`	foto precisa existir

No PUT, o painel manda também `versao` (a que tinha ao abrir). Se não bater com a do banco, 409 com o nome de quem salvou antes. Sem `versao` (o interruptor "No site"), vale o último.

Nas listas de foto, o GET e o item devolvido trazem `foto_url` e `foto_alt`, e o nome do item na auditoria é a descrição da foto.

A ordem precisa ter exatamente os itens que existem; se alguém criou ou apagou um item no meio, 409.

Fotos

MÉTODO	ROTA	O QUE FAZ
GET	`/admin/midias`	a biblioteca, sem os bytes, com quantos lugares usam cada foto
POST	`/admin/midias`	multipart: `arquivo`, `alt`, `largura`, `altura`
PUT	`/admin/midias/:id`	`{ "alt" }` corrige a descrição
DELETE	`/admin/midias/:id`	apaga se nada usar; foto original nunca

Upload: só JPG, PNG e WebP, conferidos pelos primeiros bytes, até 1,5 MB. Descrição de 3 a 200 caracteres.

Textos do site

`GET /admin/textos` → `{ valores: { chave: valor }, fotos: { chave: url } }`

`PUT /admin/textos` com `{ "valores": { "hero.lema": "…", "sobre.foto": "12", "rodape.frase": null } }`. `null` volta o campo ao original. As chaves válidas e os limites estão em `src/lib/textos.ts`. Um campo inválido barra o pacote inteiro. Em campo de uma linha, quebra de linha vira espaço; em campo de várias linhas, linhas vazias somem.

Contatos e mapa

`GET /admin/configuracoes` → `{ config: { whatsapp, instagram, mensagem, latitude, longitude } }`

`PUT /admin/configuracoes` com qualquer combinação de:

CAMPO	ACEITA	GRAVA
`whatsapp`	`(63) 98511-6783`, com ou sem 55	`5563985116783`
`instagram`	`@stewer.gabriel` ou o link do perfil	`stewer.gabriel`
`mensagem`	3 a 200 caracteres	espaços e quebras juntados
`coordenadas`	`-10.18256, -48.32910` como o Google Maps copia, ou o link longo de um lugar	`latitude` e `longitude`, só se o ponto for no Brasil

Tudo junto ou nada.

`POST /admin/link-mapa` `{ url }` → `{ latitude, longitude }`: abre um link do Google Maps e devolve o ponto. Link longo é lido direto, sem rede. Link curto (`maps.app.goo.gl`, do botão Compartilhar) é seguido pelo servidor: só para hosts do Google, só HTTPS, sem usuário nem porta, até 6 redirecionamentos e 5 s. Qualquer outro endereço dá 400 sem nenhum acesso (proteção contra SSRF).

Usuários

MÉTODO	ROTA	O QUE FAZ
GET	`/admin/usuarios`	quem tem acesso, com o último login
POST	`/admin/usuarios`	`{ nome, email }` → item e `senhaTemporaria` (aparece uma vez)
PUT	`/admin/usuarios/:id`	gera senha temporária nova (esqueceu a senha) e encerra as sessões abertas da pessoa
DELETE	`/admin/usuarios/:id`	tira o acesso; não vale para si mesmo nem para o último

Outros

MÉTODO	ROTA	O QUE FAZ
PUT	`/admin/senha`	`{ atual, nova }`; quem está com senha temporária não precisa da atual. Mínimo 10 caracteres. Encerra as outras sessões da pessoa
GET	`/admin/auditoria?de=…&ate=…[&usuario=id]`	registros do período, do mais novo ao mais velho (`usuario` que não é número dá 400)
GET	`/admin/espaco`	`{ usado, limite, porcentagem, bytesFotos, fotosEnviadas }`

CAPÍTULO 05

Plano e relatório de testes

Caixa branca, cinza e preta, segurança, desempenho, rastreabilidade e achados.

Como o sistema é verificado, o que cada suíte cobre e como cada requisito de requisitos.md é testado.

Resumo

SUÍTE	TÉCNICA	O QUE RODA	TESTES
Unitários	caixa branca	bibliotecas da API e do site, isoladas	233
Banco	caixa branca	migrations aplicadas do zero num SQLite em memória	53
API	caixa preta e caixa cinza	a API real (wrangler, runtime da Cloudflare) com um D1 isolado	210
Segurança	caixa preta e caixa cinza	OWASP Top 10 contra a API real: SQL injection, sessão, cabeçalhos, SSRF, upload	318
Desempenho	caixa branca	orçamento de peso do build (`dist/`)	16
E2E	caixa preta	site e painel num navegador, no computador (1440px) e no celular (390px)	122
Total automatizado			**952**
Carga e estresse	caixa preta	autocannon contra a API real, até 100 conexões	7 cenários
Lighthouse	caixa preta	celular e computador, três rodadas cada, na CI	2 telas

Os E2E rodam duas vezes cada, uma no computador e uma no celular: são 61 cenários (dois deles valem só para uma das telas e pulam a outra). Entre os E2E estão os de acessibilidade (axe-core, WCAG 2 A e AA), os de segurança no navegador, os Core Web Vitals, a abertura de marca e o movimento de rolagem.

Cobertura de linhas dos unitários: 88% (98% em `src/lib`, 80% em `functions/_lib`). O que fica de fora do unitário depende do banco (sessões, `conteudo.ts`, `auditoria.ts`) e é coberto pelas suítes de API e segurança.

As três técnicas

Caixa branca conhece o código e testa os caminhos de dentro: cada ramo da validação, o hash de senha, a assinatura do token, a leitura das coordenadas, a troca do conteúdo de reserva, o formato binário do zip. Fica em `tests/unit/` e `tests/db/`.

Caixa preta só conhece a interface: manda um pedido HTTP ou clica na tela e confere a resposta, sem saber como foi feito. É a maior parte de `tests/api/` e todo o `tests/e2e/`. Os contratos em `tests/contratos.ts` (schemas Zod) definem o que a API promete ao site e ao painel.

Caixa cinza faz o pedido pela interface, como a preta, mas depois abre o banco do servidor de teste (só leitura) e confere o que foi gravado: que a senha virou hash, que a auditoria registrou quem fez, que a versão subiu, que o dado inválido não chegou ao banco, que nenhuma tabela mudou depois de 215 tentativas de SQL injection, que "Sair" apagou a sessão. Os testes marcados "caixa cinza" em `tests/api/` e `tests/seguranca/` fazem isso pela função `banco()` de `tests/api/cliente.ts`.

Além das três técnicas, duas frentes têm suíte própria:

Segurança ataca a API e o navegador como alguém mal-intencionado que conhece o sistema (inclusive com o `JWT_SECRET` vazado), organizada pelo OWASP Top 10. O resumo por risco está em `SECURITY.md`.

Desempenho mede em três camadas: o peso do que o build gera (orçamento), o que o navegador sente com a rede e a CPU limitadas (Core Web Vitals, como o Lighthouse) e o quanto a API aguenta (carga e estresse).

Ambiente

Nenhum teste toca o banco de desenvolvimento nem o de produção. `tests/servidor-teste.mjs` cria, a cada rodada, uma pasta temporária, aplica todas as migrations, cria um usuário de teste e sobe o `wrangler pages dev` apontado para ela. Os testes de API e os E2E usam esse servidor; ao terminar, a pasta é apagada.

Os arquivos das suítes de API e segurança contam uma história em ordem (01-contrato até 13-entradas) sobre um banco só, por isso rodam em sequência, pelo nome. Os E2E que mudam conteúdo devolvem tudo como estava, porque o computador e o celular rodam sobre o mesmo banco.

Como rodar

```
npm run build            # a API e os E2E servem o dist/
npm test                 # unitários + banco + API + segurança + orçamento de peso
npm run test:e2e         # E2E (baixe o navegador uma vez: npx playwright install chromium)
npm run test:seguranca   # só a segurança: API (OWASP) e navegador
npm run test:desempenho  # só o desempenho: orçamento e Core Web Vitals
npm run test:carga       # carga e estresse da API (gera tests/relatorio-carga/)
npm run auditoria        # npm audit das dependências
npm run test:tudo        # tudo, na ordem
npm run test:cobertura
```

No GitHub, três workflows rodam a cada push e pull request na `main`:

- `testes.yml`: auditoria de dependências, tipos, orçamento de peso, unitários, banco, API, segurança, carga e estresse e E2E; guarda o relatório do E2E quando algo falha e publica a tabela de carga no resumo do job;
- `lighthouse.yml`: Lighthouse no celular e no computador, com os tetos de `tests/lighthouse`;
- `codeql.yml`: análise estática de segurança do GitHub (também toda segunda-feira).

O Dependabot abre pull request quando sai atualização de dependência.

O que cada suíte cobre

Unitários (`tests/unit/`)

ARQUIVO	TESTES	O QUE CONFERE
`auth.test.ts`	34	hash PBKDF2 com salt, compatível com o script de criar acesso; JWT assinado, adulterado, vencido, `alg: none`, outro algoritmo, carga fora do formato; cookie seguro
`http.test.ts`	26	leitura do corpo com limite e tipo; formato único de erro; erro inesperado sem detalhe; id de rota só com dígitos (`0x1`, `1e3`, `Infinity` recusados)
`entidades.test.ts`	32	lista fechada de tabelas (inclusive `__proto__` e `toString`); limites; enquadramento da foto; o que mudou, em português
`config.test.ts`	38	WhatsApp, Instagram, mensagem e coordenadas no formato que o Stewer digita ou cola, inclusive o link longo do Google Maps; link curto pede para ser aberto; ponto fora do Brasil
`coordenadas.test.ts`	27	o ponto em cada formato de link do Google Maps (alfinete, `?q=`, `/search/`, `@`); prioridade entre eles; vírgula decimal; arredondamento
`link-mapa.test.ts`	16	o resolvedor do link curto: só hosts do Google, só HTTPS, sem usuário nem porta, no máximo 6 saltos, página de consentimento; nada vai para a rede (fetch falso)
`mapa.test.ts`	9	a ordem das fontes do mapa (CARTO, OpenFreeMap, OSM), quando desistir de uma, o crédito de cada uma
`movimento.test.ts`	12	o número que a contagem usa ("+800", "10+", "1.500"): o último quadro é igual ao texto original
`site.test.ts`	7	telefone formatado, links de WhatsApp, Instagram e rota; graus, minutos e segundos iguais aos do site original
`textos.test.ts`	10	chaves únicas; todas as seções; texto original cabe no limite; aplicar ignora chave e tipo errados; só o endereço é editado na página de contatos
`conteudo-remoto.test.ts`	12	troca antes do render, mesma referência das listas, vídeos intactos, foto quebrada fora, abertura sem foto, API fora ou lenta
`utilitarios.test.ts`	10	zip do backup lido byte a byte e conferido com o CRC do zlib; nomes de arquivo; senha temporária

Banco (`tests/db/`)

`schema.test.ts` (53): migrations em sequência e aplicáveis uma a uma; todas as tabelas; o seed igual ao conteúdo do código, item a item e na mesma ordem; toda foto do seed existe em `public/`; CHECK, NOT NULL, UNIQUE e FOREIGN KEY de cada lista; sessão de pessoa inexistente recusada e sessões apagadas junto com a pessoa; versão para edição simultânea; índices e o plano da consulta da auditoria.

API (`tests/api/`)

ARQUIVO	TESTES	O QUE CONFERE
`01-contrato`	10	`/api/conteudo` no contrato, sem nada interno, `no-cache`; formato único de erro; foto original redireciona
`02-autenticacao`	64	login, trava, hash no banco, sessão, logout, e as 51 rotas do painel exigindo login
`03-listas`	42	as sete listas no contrato; criar, editar, ocultar, reordenar, apagar refletindo no site; ordem incompleta; validação; lista fechada
`04-fotos`	18	tipos aceitos pelos bytes; arquivo disfarçado, SVG e grande demais recusados; download; foto em uso protegida (lista e texto)
`05-contatos`	22	WhatsApp, Instagram, mensagem e mapa chegando ao site; link do Google Maps (longo e curto); inválidos; pacote inteiro ou nada
`06-textos`	13	gravar, foto de seção, voltar ao original, normalização, limites, pacote inteiro ou nada
`07-concorrencia`	6	versão velha recusada com o nome de quem salvou; dez gravações ao mesmo tempo, uma vence
`08-usuarios-auditoria`	18	dar acesso, senha temporária (encerra a sessão aberta), troca obrigatória, tirar acesso, auditoria, espaço do banco
`09-seguranca`	17	CSRF, XSS guardado como texto, Instagram malicioso, corpo grande, tipo errado, prototype pollution, erro sem vazar detalhe

Segurança (`tests/seguranca/` , OWASP Top 10)

ARQUIVO	TESTES	O QUE CONFERE
`10-sql-injection`	215	15 payloads (tautologia, `UNION`, comando empilhado, comentário, blind por tempo, `PRAGMA`, `sqlite_master`) em login, rotas, ids, textos, contatos, auditoria, usuários e link do mapa; nunca 500, nunca dado do banco na resposta; caixa cinza: nenhuma tabela mudou e o banco continua íntegro
`11-sessao`	23	token forjado: `alg: none`, carga trocada, outro segredo, vencido, `JWT_SECRET` vazado sem sessão aberta ou com a sessão de outra pessoa; cookie malformado; "Sair" mata o cookie copiado e não derruba o outro aparelho; troca de senha e senha temporária encerram as outras sessões; acesso removido vale na hora
`12-cabecalhos`	29	CSP diretiva a diretiva (nada de `*`, `unsafe-eval` ou origem a mais), `nosniff`, `X-Frame-Options`, HSTS, COOP, Permissions-Policy nas páginas e na API, inclusive em erro e em foto; o build sem script inline; painel fora do Google; rota de API inexistente em JSON; CORS fechado
`13-entradas`	51	SSRF no link do mapa (metadados da nuvem, localhost, rede interna, `file:`, `user@`, porta, domínio parecido); upload de HTML, SVG, executável e PDF disfarçados; poliglota servido como imagem que não executa; header injection no nome do download; path traversal; mass assignment

No navegador, `seguranca.spec.ts` confere que o navegador obedece: script injetado e script de outro site não rodam, o painel não abre em iframe de outro site, o cookie é invisível ao JavaScript e não vale depois de "Sair", e XSS guardado aparece como texto. E todo E2E tem um **fiscal da CSP**: qualquer coisa que a política bloquear durante qualquer teste reprova o teste, para a CSP nunca quebrar o site em silêncio.

Desempenho

Orçamento de peso (`tests/desempenho/orcamento.test.ts` , 16): confere o `dist/` , o que o celular baixa de verdade, em gzip.

O QUÊ	TETO	HOJE
JavaScript da abertura do site	65 KB	~58 KB
CSS da abertura do site	9 KB	~6 KB
HTML	2 KB	~1 KB
JavaScript da abertura do painel	72 KB	~63 KB
foto de `public/`	250 KB cada	maior: 217 KB

Também: o mapa e o painel fora da abertura do site; nada de outro servidor bloqueando a primeira pintura; o MapLibre como único arquivo grande (e sob demanda); só fontes do subconjunto latino.

Core Web Vitals (`tests/e2e/desempenho.spec.ts`): no navegador, com as condições do Lighthouse (celular: 4G lento, 150 ms e 1,6 Mbps, CPU 4× mais lenta; computador: cabo), como um visitante novo, com a abertura de marca e o movimento ligados.

MÉTRICA	TETO (O "BOM" DO GOOGLE)	CELULAR	COMPUTADOR
FCP	1,8 s	~0,7 s	~0,25 s
LCP	2,5 s	~1,7–2,2 s	~0,5 s
TBT	200 ms	~70 ms	0 ms
CLS (rolando a página toda)	0,1	0	0
INP (abrir uma foto)	200 ms	~40 ms	~15 ms
p95 de `/api/conteudo`	300 ms	~15 ms	~15 ms
painel do login à tela pronta, no 4G lento	3 s	~2 s	~2 s

Também: o Leaflet só baixa perto do mapa e nenhum pedido sai para outro servidor na abertura.

Carga e estresse (`tests/carga/carga.mjs` , autocannon), contra o servidor de teste (um processo só, na máquina da CI: os números da Cloudflare são melhores; isto pega regressão, não promete latência).

CENÁRIO	CONEXÕES	PEDIDOS/S	P50	P97,5	5XX
conteúdo do site	10	~380	24 ms	41 ms	0
página inicial	10	~370	25 ms	45 ms	0
painel (com login)	5	~270	16 ms	40 ms	0
estresse	25	~440	51 ms	97 ms	0
estresse	50	~440	102 ms	233 ms	0
estresse	100	~425	221 ms	326 ms	0
força bruta: 200 senhas erradas na mesma conta	10	—	36 ms	243 ms	0

Lighthouse (CI, três rodadas por tela): hoje 95 / 100 / 100 / 100 no celular e 98 / 100 / 100 / 100 no computador (desempenho, acessibilidade, boas práticas, SEO). Os tetos estão em `tests/lighthouse/LEIAME.md`.

E2E (`tests/e2e/`)

ARQUIVO	CENÁRIOS	O QUE CONFERE
`site.spec.ts`	12	site abre sem erro com o banco; portfólio intercala vídeo; foto ampliada pelo teclado; API fora e lenta; links de contato; mapa (e as reservas dele); no celular, um dedo rola a página e dois mexem no mapa; seção some sem itens; HTML no nome do lugar não vira elemento
`painel.spec.ts`	17	login; editar serviço e ver no site; reordenar com as setas; texto e voltar ao original; enviar foto (reduzida para 1600px); nova foto no portfólio; contatos; colar link do Google Maps e arrastar o alfinete; endereço; erro em português; duas pessoas no mesmo item; acesso com senha temporária; auditoria; espaço; todas as abas sem rolagem lateral; sair
`acessibilidade.spec.ts`	4	login, todas as abas e o formulário aberto sem violação grave; o que o painel coloca no site sem violação nova
`seguranca.spec.ts`	6	ver "Segurança" acima
`desempenho.spec.ts`	7	ver "Desempenho" acima
`abertura.spec.ts`	7	a abertura de marca aparece, trava a rolagem por trás, sobe sozinha em até 2 s, aparece de novo ao recarregar, espera a API lenta, não prende o site quebrado, não aparece para quem pede menos movimento
`movimento.spec.ts`	8	o GSAP só baixa quando a pessoa rola; título dividido em linhas sobe e mantém o nome acessível; foto se abre e devolve o hover; número conta até o valor e o leitor de tela lê o final; o que já estava na tela não some; parallax só no computador; movimento reduzido não baixa nada

Os E2E rodam com `prefers-reduced-motion: reduce`, para olhar o resultado e não a coreografia; `abertura`, `movimento` e `desempenho` ligam o movimento.

Rastreabilidade

REQUISITO	TESTADO EM
RF01	`conteudo-remoto` · `01-contrato` · `03-listas` · `site.spec`
RF02	`conteudo-remoto` · `03-listas` (ocultar, tudo oculto) · `site.spec` (seção some)
RF03	`conteudo-remoto` (falhas e 2,5 s) · `site.spec` (API fora e lenta)
RF04	`textos` · `06-textos` · `painel.spec`
RF05	`site` · `conteudo-remoto` · `05-contatos` · `site.spec` (links e mapa)
RF06	`conteudo-remoto` (vídeos intactos) · `site.spec` (vídeo intercalado)
RF07	`mapa` · `site.spec` (OpenFreeMap sem chave, OSM com o estilo fora) · `desempenho.spec` (mapa sob demanda)
RF08	`abertura.spec` · `movimento.spec` · `movimento` · `desempenho.spec` (CLS com movimento)
RF09	`site.spec` (mapa pela âncora) · `movimento.spec` (âncora antes do movimento)
RF21	`site.spec` (um dedo rola, aviso de dois dedos)
RF10	`02-autenticacao` · `painel.spec`
RF11	`entidades` · `03-listas` · `painel.spec`
RF12	`utilitarios` (zip) · `04-fotos` · `painel.spec`
RF13	`painel.spec` (3000px vira 1600px)
RF14	`textos` · `06-textos` · `painel.spec`
RF15	`config` · `coordenadas` · `link-mapa` · `05-contatos` · `08-usuarios-auditoria` (senha) · `painel.spec`
RF16	`utilitarios` (senha temporária) · `08-usuarios-auditoria` · `painel.spec`
RF17	`08-usuarios-auditoria` · `painel.spec`
RF18	`entidades` (o que mudou) · todas as suítes de API (`ultimaAuditoria`) · `painel.spec`
RF19	`08-usuarios-auditoria` · `painel.spec`
RF20	`03-listas` · `painel.spec`
RNF01	`auth` · `02-autenticacao` (caixa cinza) · `08-usuarios-auditoria`
RNF02	`auth` · `schema` (sessões) · `02-autenticacao` · `11-sessao` · `seguranca.spec`
RNF03	`09-seguranca` · `12-cabecalhos` · `seguranca.spec` · fiscal da CSP em todos os E2E
RNF04	`02-autenticacao`

REQUISITO	TESTADO EM
RNF05	`entidades` · `03-listas` · `09-seguranca` · `10-sql-injection`
RNF06	`04-fotos` · `13-entradas`
RNF07	`schema` · `07-concorrencia` · `painel.spec`
RNF08	`05-contatos` · `06-textos` · `03-listas` (ordem)
RNF09	`entidades` · `config` · `schema` · `03-listas` · `04-fotos`
RNF10	`contratos.ts` em todas as suítes de API · `http`
RNF11	`painel.spec` (todas as abas no celular e no computador)
RNF12	`acessibilidade.spec` · `movimento.spec` (nome acessível) · Lighthouse
RNF13	`schema` (índices) · `08-usuarios-auditoria` (limpeza e espaço)
RNF14	`conteudo-remoto` · `site.spec`
RNF15	`schema` · workflow do CI
RNF16	`schema` (seed igual ao código) · `site` (coordenadas iguais) · comparação de capturas (abaixo)
RNF17	`link-mapa` · `13-entradas` · `12-cabecalhos` · `npm audit` e CodeQL na CI
RNF18	`orcamento` · `desempenho.spec` · `movimento.spec` (GSAP fora da abertura) · Lighthouse
RNF19	`desempenho.spec` (p95 e painel) · `carga.mjs`

Achados

O que os testes encontraram durante o desenvolvimento e já foi corrigido:

ACHADO	COMO APARECEU	CORREÇÃO
Instagram `a/../../x` era aceito e gravado como `a`, sem avisar	`09-seguranca`	só limpa o resto do texto quando é o link do perfil; qualquer outra barra é recusada
Títulos dos grupos da lateral do painel com contraste abaixo do AA	`acessibilidade.spec`	cor trocada pelo tom "areia" do painel
Os números de "Quem é" quebravam diferente no celular depois da troca para lista	comparação de capturas	volta à mesma estrutura de flex do original
Os botões de zoom do mapa perdiam o estilo escuro	comparação de capturas	o CSS do Leaflet voltou a carregar antes do CSS do site
Ordem enviada sem um dos itens deixaria dois itens na mesma posição	revisão ao escrever `03-listas`	a API exige a lista completa e responde 409

ACHADO	COMO APARECEU	CORREÇÃO
"Sair" só apagava o cookie: um token copiado antes continuava entrando por 8 horas	`11-sessao`	sessões gravadas no banco (migração 0003): sair, trocar a senha e senha temporária encerram
`?usuario=abc` na auditoria virava `NaN` e o filtro sumia, devolvendo a auditoria de todo mundo	`10-sql-injection`	o filtro é validado e responde 400
O id da rota aceitava `0x1` e `1e3` como número	`13-entradas`	só dígitos
Rota de API inexistente respondia 200 com o HTML do site	`12-cabecalhos`	`/api/[[rota]]` responde 404 (ou 405) em JSON
A cortina da abertura nunca subia se o JavaScript do site falhasse ao carregar	`abertura.spec`	falha ao montar também libera a cortina
Títulos em linhas eram lidos colados por leitor de tela e pelo Google ("StewerGabriel")	`movimento.spec` (nome acessível)	espaço antes de cada quebra de linha
Link com âncora (`#onde-atende`) abria no topo, porque a seção ainda não existia	`site.spec` , depois que o mapa passou a carregar sob demanda	a página vai até a âncora depois de montar
A área do mapa tinha `aria-label` sem papel antes de o Leaflet chegar	`acessibilidade.spec`	`role="region"`
"Scroll" da abertura com contraste 1,7:1	Lighthouse	cor areia (5,2:1)
`robots.txt` inexistente: o endereço devolvia o HTML e o Google lia 36 erros	Lighthouse	`robots.txt` e `sitemap.xml`
LCP de 3,2 s no celular: as três fotos da abertura e as fontes dividiam a banda, e a foto só era pedida depois de o React montar	`desempenho.spec`	primeira foto com prioridade e as outras depois; conteúdo pré-carregado com o HTML; entrada sem React (LCP ~1,7 s)
Fontes vinham de um CSS bloqueante do Google Fonts	`orcamento`	fontes servidas pelo próprio site
Leaflet (50 KB) na abertura para um mapa no fim da página	`orcamento`	mapa sob demanda
Foto do portfólio com 871 KB	`orcamento`	JPEG de 96 KB (migração 0004)
O GSAP travava o processador do celular por ~250 ms logo depois de carregar	`desempenho.spec` (TBT)	só baixa quando a pessoa rola, e se prepara em pedaços
Montar a página inteira de uma vez travava ~200 ms, congelando o S da abertura	`desempenho.spec` (TBT)	o resto da página monta numa transição do React
No celular, descer a página com o dedo sobre o mapa arrastava o mapa	uso no celular (relatado), agora coberto em `site.spec`	um dedo rola a página, dois mexem no mapa, com aviso

Fidelidade ao design (RNF16)

Além dos testes automáticos, o site novo foi comparado com o site da `main` por captura da página inteira, no computador (1440px) e no celular (390px), pixel a pixel. As únicas diferenças restantes são o quadro do vídeo em reprodução, a foto da abertura no meio da troca e o endereço do rodapé, que agora mostra "Nº 15" como o cartão do mapa, porque os dois leem o mesmo endereço.

CAPÍTULO 06

Segurança

Os controles pelo OWASP Top 10 e onde cada um é testado.

Como avisar de uma falha

Achou uma vulnerabilidade? Não abra uma issue pública. Use o aviso privado de vulnerabilidade do GitHub, com o passo a passo para reproduzir. A resposta sai em até 7 dias.

Superfície

PARTE	QUEM ACESSA	O QUE GUARDA
site (`/`)	qualquer pessoa	nada: só lê `/api/conteudo`
painel (`/admin`)	só quem tem acesso, com senha	sessão em cookie `HttpOnly`
API (`/api/admin/*`)	só o painel, da mesma origem	conteúdo do site, fotos, pessoas com acesso, auditoria
banco (Cloudflare D1)	só a API, por binding	nunca exposto na internet

Controles, pelo OWASP Top 10 (2021)

Cada controle tem teste automatizado; a CI roda tudo a cada push.

RISCO	CONTROLE	TESTADO EM
A01 Controle de acesso	toda rota do painel exige sessão; acesso removido vale na hora; senha temporária só libera a troca; id de rota só aceita dígitos	`02-autenticacao` (50 rotas), `11-sessao`, `13-entradas`
A02 Criptografia	PBKDF2-SHA256 com 100 mil iterações e salt por pessoa; comparação em tempo constante; HSTS	`auth.test`, `02-autenticacao`, `12-cabecalhos`
A03 Injeção	SQL sempre por bind, tabelas de lista fechada; React escapa todo texto; CSP sem script inline	`10-sql-injection` (215 casos), `09-seguranca`, `seguranca.spec`
A04 Design inseguro	trava de 5 tentativas por e-mail; mesma mensagem para e-mail inexistente; concorrência por versão	`02-autenticacao`, `07-concorrencia`

RISCO	CONTROLE	TESTADO EM
A05 Configuração	CSP, `nosniff`, `X-Frame-Options`, COOP, Permissions-Policy; painel fora do Google; rota de API inexistente responde 404 em JSON	`12-cabecalhos`, `seguranca.spec`, fiscal da CSP em todos os E2E
A06 Componentes vulneráveis	`npm audit` bloqueia a CI; Dependabot semanal; CodeQL	`.github/workflows`
A07 Autenticação	JWT só HS256 (recusa `alg: none`); sessão gravada no banco, encerrada por "Sair", troca de senha e senha temporária; cookie `HttpOnly` + `SameSite=Strict`	`auth.test`, `11-sessao`, `seguranca.spec`
A08 Integridade	campo que o painel não manda não é gravado (mass assignment); upload conferido pelos bytes	`13-entradas`, `04-fotos`
A09 Registro	auditoria de quem mudou o quê, com data; tentativas de login com IP	`08-usuarios-auditoria`
A10 SSRF	o link do mapa só é aberto se for do Google, por HTTPS, sem usuário nem porta, no máximo 6 saltos	`link-mapa.test`, `13-entradas`

CSRF: o cookie é `SameSite=Strict` e, além disso, todo pedido que muda dado precisa vir da mesma origem (`09-seguranca`).

Fora do alcance

- Ataques de volume (DDoS): ficam com a Cloudflare, na frente de tudo.
- Quem já tem a senha do painel: por isso existe a auditoria e o "Sair".
- O `JWT_SECRET` vazado não basta para entrar: o token precisa apontar para uma sessão aberta no banco (`11-sessao`).

Rodar só a segurança

```
npm run build
npm run test:seguranca   # API (OWASP) + navegador
npm run auditoria        # dependências
```

CAPÍTULO 07

Manual do painel

Como o Stewer usa o painel, em linguagem simples.

O painel é onde você muda o seu site sem precisar de ninguém. Tudo que você salvar aparece no site na próxima vez que a página for aberta.

O painel fica em **stewergabriel.com/admin**. Funciona no celular e no computador. Dica: no celular, abra o endereço e use "Adicionar à tela inicial", que ele vira um atalho como um app.

Entrar

1. Abra **stewergabriel.com/admin**.
2. Digite o seu e-mail e a sua senha e toque em **Entrar**.

Errou a senha cinco vezes seguidas? O painel trava esse e-mail por 15 minutos, por segurança. É só esperar.

A sessão dura 8 horas. Depois disso o painel pede a senha de novo.

Primeira vez

Se alguém criou o seu acesso, você recebeu uma **senha temporária** (algo como `kq7m-x3ra-p9ve`). Entre com ela: o painel vai pedir para você criar uma senha sua, com pelo menos 10 caracteres. Depois disso a temporária não vale mais.

Esqueci a senha

Peça para outra pessoa que tem acesso ao painel abrir **Usuários** e tocar em **Esqueceu a senha** no seu nome (e tocar de novo para confirmar). Ela te manda a senha nova pelo WhatsApp (o painel já deixa a mensagem pronta para copiar).

Se ninguém mais tiver acesso, fale com o Gustavo: ele recupera pelo terminal.

Como o painel é organizado

Do lado esquerdo (no celular, na faixa de cima, que rola de lado) ficam as partes do painel:

PARTE	O QUE MUDA NO SITE
Textos do site	todo texto fixo: abertura, quem é, títulos das seções, endereço, horário, contato, rodapé, e a foto de "Quem é"
Fotos da abertura	as fotos que se revezam ao lado do seu nome
Números	+800 clientes, 10+ anos...
Serviços	os cards numerados da seção Serviços
Consultoria	os itens da Consultoria de Imagem
Ensino	o que se aprende nas aulas
Fotos das aulas	a galeria "Bastidores das aulas"
Portfólio	as fotos de cortes
Contatos e mapa	WhatsApp, mensagem pronta, Instagram e o ponto do mapa
Todas as fotos	a biblioteca com todas as fotos do site
Usuários	quem entra no painel
Auditoria	o que foi feito, por quem e quando
Minha conta	trocar a sua senha e ver o espaço usado

Em quase toda página tem o botão **Ver no site**, que abre a seção certa em outra aba.

Textos do site

1. Abra **Textos do site**.
2. Escolha a seção na faixa de cima (Abertura, Quem é, Serviços...).
3. Mude o que quiser. Embaixo de cada campo aparece quantas letras cabem.
4. Toque em **Salvar** no fim da seção.

Alguns campos são de "uma linha por linha", como o seu nome grande na abertura ou o endereço. Neles, cada linha que você escreve vira uma linha no site.

Mudou e não gostou? Toque em **voltar ao original** em cima do campo e salve: volta o texto que o site tinha antes do painel.

Listas (serviços, consultoria, ensino, números)

Cada lista mostra os itens na mesma ordem do site.

- **Mudar a ordem:** use as setinhas ↑ e ↓ do lado esquerdo. Em Serviços, o número do card (01, 02, 03) segue essa ordem.
- **Esconder sem apagar:** desligue a chave **No site**. O item some do site mas continua guardado. Ligue de novo quando quiser.
- **Editar:** toque no nome do item. Abre um painel do lado; mude e toque em **Salvar**.
- **Novo:** toque em **+ Novo serviço** (ou novo item, novo tópico...).
- **Apagar:** abra o item e toque em **Apagar**. O painel pede para tocar de novo, para confirmar. Apagar não tem volta; se tiver dúvida, esconda.

Se você esconder todos os serviços, a seção some do site inteira. Nas outras listas, some só aquele bloco.

"Alguém salvou enquanto você editava"

Esse aviso aparece quando duas pessoas (ou você no celular e no computador) abrem o mesmo item e salvam. O painel não deixa uma mudança apagar a outra sem querer: feche, abra o item de novo para ver como ficou e refaça a sua mudança.

Fotos da abertura, das aulas e do portfólio

1. Toque em **+ Nova foto**.
2. Escolha uma foto que já está no site, ou toque em **Enviar uma foto nova** e pegue do celular.
3. Escreva uma descrição curta da foto (por exemplo, "Degradê com risca à navalha"). Ela é lida para quem não enxerga e ajuda o Google a entender a foto.
4. Toque em **Enviar foto**. Ela entra no fim da lista e já aparece no site.

Para mudar a parte da foto que aparece (cortar mais em cima ou mais embaixo), toque na foto da lista e arraste a barra de **Enquadramento**.

O painel diminui a foto antes de enviar. Uma foto de celular de 5 MB vira uma de uns 300 KB, sem perder qualidade na tela. Não precisa editar antes.

Vídeos não sobem pelo painel: são grandes demais para o banco do site. Para trocar o vídeo do portfólio, mande para o Gustavo.

Todas as fotos

A biblioteca mostra todas as fotos do site. Tocando numa foto, você pode:

- corrigir a descrição;
- baixar a foto;
- apagar, se ela não estiver em uso em nenhum lugar do site.

As fotos que vieram com o site original não podem ser apagadas pelo painel (só tiradas das listas). **Baixar todas** junta todas as fotos num arquivo .zip: é uma boa cópia de segurança para guardar de vez em quando.

Contatos e mapa

- **WhatsApp:** com DDD, do jeito que você escreve, por exemplo (63) 98511-6783. É usado em todos os botões de WhatsApp e no telefone do site.
- **Mensagem que já vem escrita:** o cliente toca no botão e o WhatsApp abre com essa mensagem pronta.
- **Instagram:** só o nome de usuário (@stewer.gabriel) ou o link do perfil.

Toque em **Salvar contatos** para gravar os três.

Endereço: o nome do lugar, o endereço, o horário e o atendimento que aparecem no cartão do mapa. Uma linha por linha, como aparece no site.

Local no mapa: se você mudar de lugar,

1. abra o lugar novo no Google Maps;
2. toque em **Compartilhar** e em **Copiar link**;
3. cole no campo **Link do Google Maps ou coordenadas**. O alfinete dourado já pula para o lugar;
4. se precisar acertar, arraste o alfinete (ou toque no mapa onde ele deve ficar). **Conferir no Google Maps** abre o ponto para você comparar;
5. toque em **Salvar local**. Se mudou de ideia antes de salvar, **Desfazer**.

Também dá para colar as coordenadas (segure o dedo em cima do lugar no Google Maps e toque nos números que aparecem).

Usuários

Para dar acesso a alguém, escreva o nome e o e-mail e toque em **Criar acesso**. O painel mostra uma senha temporária uma vez só: copie a mensagem e mande para a pessoa.

Para tirar o acesso, toque em **Tirar acesso** no nome dela. A pessoa sai na hora, mesmo se estiver com o painel aberto. Ninguém consegue tirar o próprio acesso, e o painel sempre fica com pelo menos uma pessoa.

Auditoria

Mostra tudo que foi feito no painel num dia: quem entrou, o que mudou e a que horas. Dá para filtrar por pessoa. Fica guardado por um ano.

Minha conta

Aqui você troca a sua senha (precisa da atual) e vê quanto espaço o site já usa no banco. O plano gratuito cabe mais de mil fotos; o painel avisa no topo se passar de 70%.

Trocar a senha desconecta o painel de todos os outros aparelhos (o celular esquecido logado, por exemplo); o aparelho em que você trocou continua entrando. Tocar em **Sair** encerra o acesso daquele aparelho de verdade: para entrar de novo, só com a senha.

Quem cuida do quê

VOCÊ, PELO PAINEL	O GUSTAVO, PELO CÓDIGO
textos, fotos, listas, contatos, mapa, acessos	vídeos, cores, fontes, layout, seções novas, domínio